FROM NOTEBOOK TO FACTORY

Deployment & Sim-to-Real

Week 13 — Everything That Breaks When You Ship

Domain shift • Monitoring • Export & latency

Last Week → This Week

WEEK 12 RECAP

- An agent learned a policy from reward alone
- Wheel slip changed which route was optimal
- Exploration has to come from somewhere
- A learned policy is only valid where it has data

TODAY

- That last point, taken seriously
- The sim-to-real gap, measured
- Domain randomisation — and its limits
- Why confidence will not save you
- Export, latency and what to optimise

The Last Mile



Twelve weeks of models, every one judged on a held-out set drawn from the same pile of data as the training set.

DEVELOPMENT

Train and test data come from one pool, split at random. The test set is a fair sample of the training distribution.

This is correct practice. It is also the only thing you have ever measured.

DEPLOYMENT

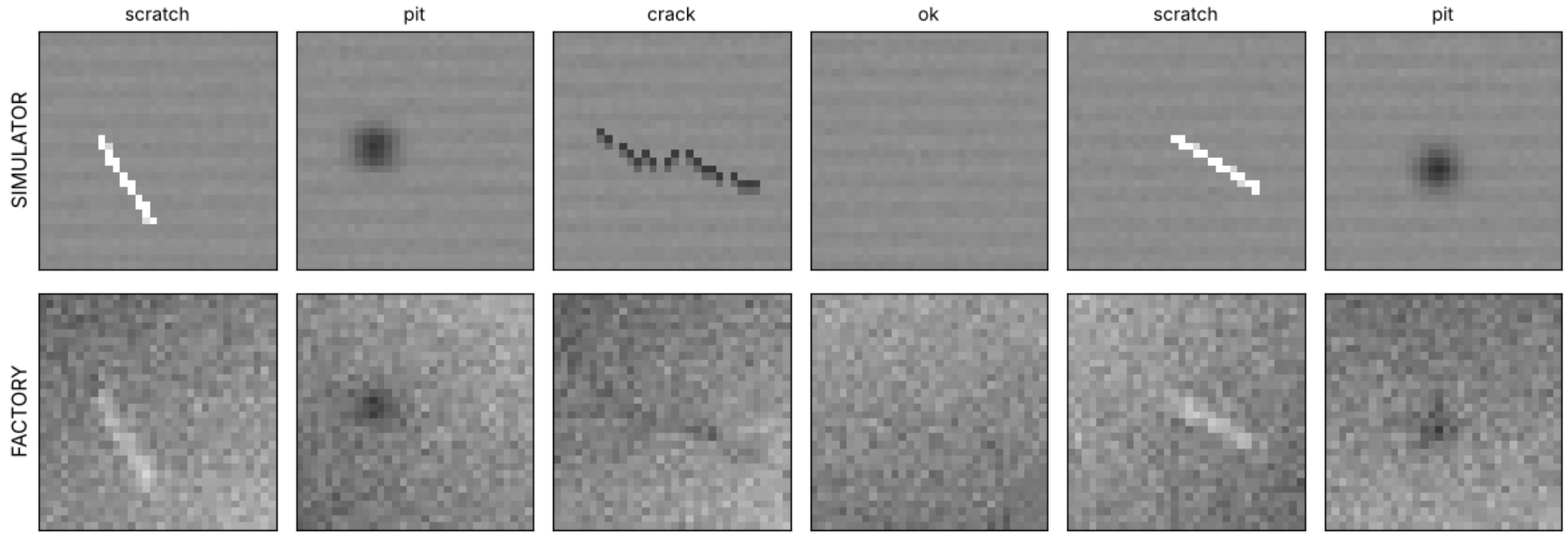
Data comes from a different physical process: another camera, another lamp, a lens, a machine that has worn.

Nothing in your validation has ever tested this.

Today: build a defect classifier that is perfect in simulation and useless on the factory floor — then work out what to do about it.

Today's Problem: Two Domains

Each column is the same part — as the simulator renders it, and as the factory camera sees it.



Anticipated

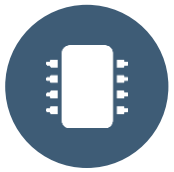
uneven lighting · gain & black level · sensor noise

Never thought of

defocus blur · a few degrees of mount angle

Hold on to that split. Which corruptions you anticipated decides the result of the entire lecture.

Perfect in Simulation, Useless on the Floor



A small CNN, trained on 2,500 clean simulated images. The kind you would actually put on an inspection station.

1.000

accuracy on the
SIMULATED test set

0.267

accuracy on the
REAL test set

0.250

chance, with
four classes

You would have reported a perfect validation score to your supervisor.

Accuracy alone does not tell you HOW it fails. On real images the model predicted “crack” for 584 of 600 parts — it collapsed onto one class.

The Obvious Fix That Does Not Work



Simulated data is free. So collect four times as much — 10,000 images instead of 2,500.

training set	simulated test	REAL test
clean sim, 2,500	**1.000**	**0.267**
clean sim, 10,000	**1.000**	**0.278**

Four times the data bought nothing. The problem was never a shortage of examples.

More data from the wrong distribution is still the wrong distribution. This is the expensive mistake: months of collection that could not possibly have helped.

Domain Randomisation



If you cannot make the simulator match reality, make it refuse to commit to any one appearance.



Vary what you are unsure of

Every training image gets a different lamp angle, gain, noise level, focus and mount angle.



The model must ignore it

No single appearance is reliable, so the only stable signal left is the shape of the defect itself.



Reality becomes one more sample

If the real camera falls inside the range you trained over, it is nothing special.

Two versions in the lab: NARROW randomises lighting, gain and noise. WIDE adds the two we never thought of — blur and rotation.

Measured: the Gap Closes in Proportion to Coverage



Randomisation only helps for the corruptions you actually randomise over. Narrow fixes the part of the shift it covers and leaves the rest; wide recovers most of the loss.

Note the cost: wide randomisation never reaches the simulated score. Tolerating blur and rotation costs accuracy on clean images — a trade worth taking.

Before You Believe That Result



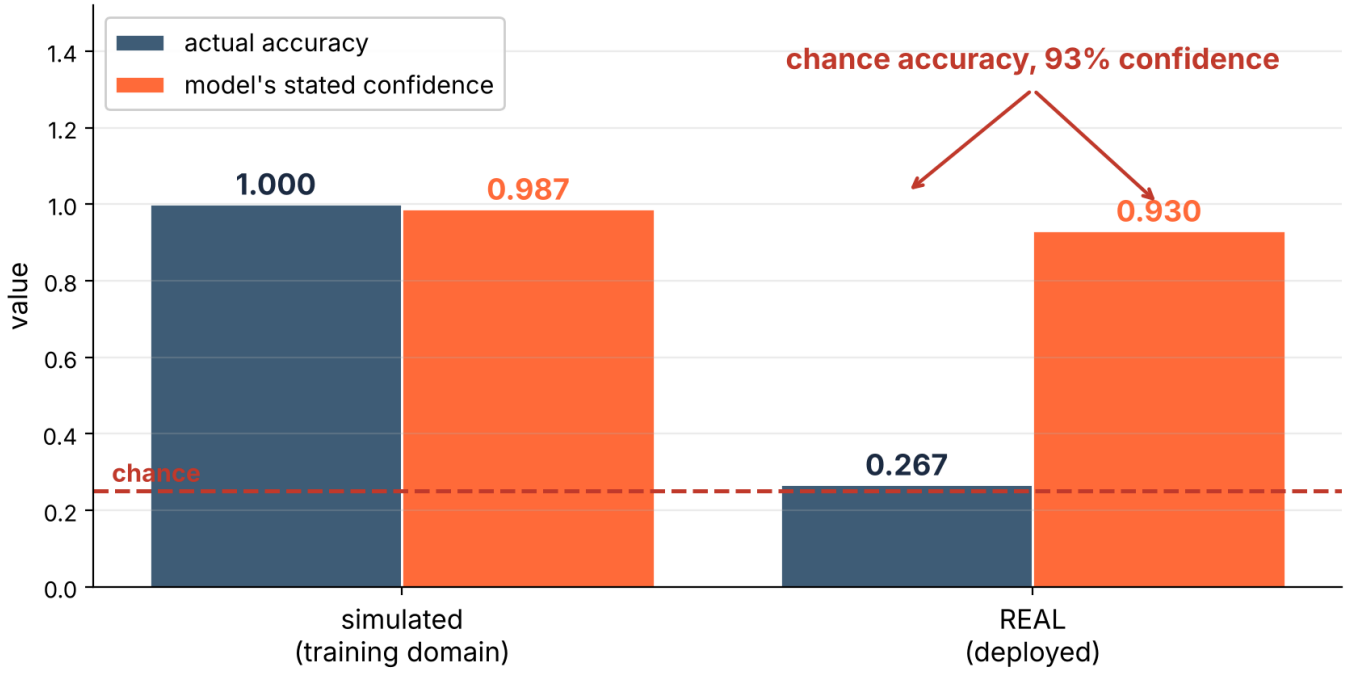
We built the factory camera ourselves, so we knew exactly which corruptions to add. A real project never hands you that.

What we did	Randomised over blur and rotation because we knew the real camera had them.
What you must do	Randomise wider than seems reasonable, over anything you cannot pin down.
What never changes	Validate on real data before trusting any of it. Some of the gap will remain.

A simulator you tuned to match your test set is just a training set with extra steps.

Does the Model Know It Is in Trouble?

If confidence dropped when the model went out of its depth, deployment would be easy: gate on confidence and route the rest to a human.



79.5%

of REAL predictions made with over 90% confidence

73.3%

of REAL predictions are wrong

0.924

mean confidence on the predictions that are WRONG

It is confidently, catastrophically wrong. A confidence threshold is not a safety mechanism against domain shift.

Why Confidence Cannot See a Domain Shift



This is structural, not a quirk of this network. Look at what the softmax actually normalises over.

softmax answers: “which of my four classes fits best?”

It never answers: “does this input resemble anything I was trained on?”

It is a relative score

The four probabilities are forced to sum to one. Something always wins, however strange the input.

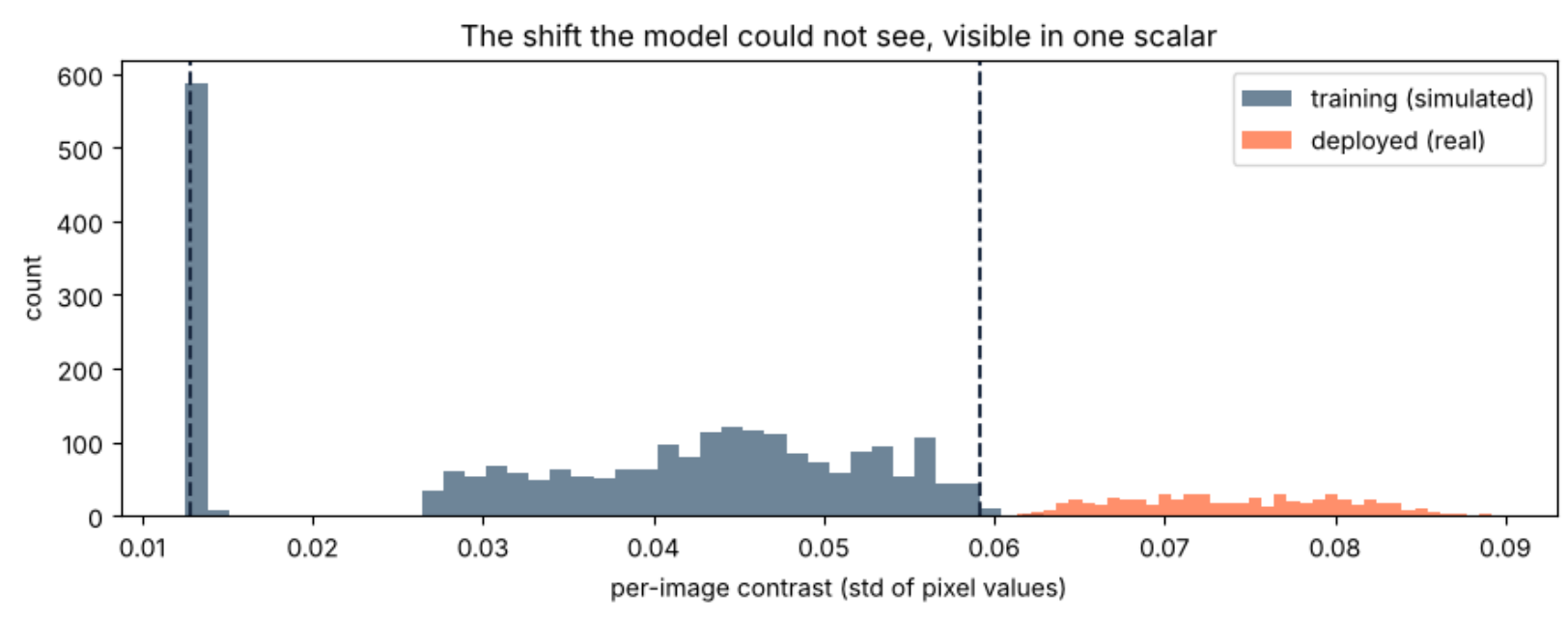
Far from the data, it gets worse

Push an input well outside training and one logit typically dominates — so confidence goes UP, not down.

Any deployment design that leans on a confidence threshold to catch domain shift is resting on nothing.

Monitor the Input, Not the Model's Opinion

The shift was obvious in the raw pixels. Build an accept band from training data alone, then flag anything outside it.



99.8%

of REAL images flagged

1.7%

of in-domain images flagged

No labels. One number per image. Works from day one.

In production, track several: per-image mean and contrast, the prediction distribution, and the rate of the rarest class. A collapse like slide 5 shows up instantly.

Shipping It: Export



torch.export captures the model as a standalone program — no Python class definition needed on the robot.

```
batch_dim = torch.export.Dim("batch")
ep = torch.export.export(model, (torch.randn(2,1,32,32),), dynamic_shapes={"x": {0: batch_dim}})
```

14,340

parameters

110 KB

exported program

0.00e+00

max |eager − exported|

The trap that costs an afternoon: export with a batch of 1 and PyTorch bakes in batch=1 — the exported program then refuses every other batch size at runtime.

Latency Against a Real Budget

“0.3 ms” means nothing until you compare it with the machine's cycle time. 120 parts/min = a 500 ms budget. A 1280×1024 part is 1,280 patches.

	ms per patch	ms per part	budget used
one patch at a time	≈ 0.34	≈ 440	≈ 90%
batched (32)	≈ 0.13	≈ 170	≈ 35%

Batching alone: about 2.5× faster, for free, and no longer running at 90% of the cycle time.

Per patch it looked fast. Per part it very nearly did not fit. Always measure at the unit the machine actually cares about.

Absolute timings vary by machine; the ratio is the stable part.

Does Quantisation Earn Its Place?



Store weights as 8-bit integers instead of 32-bit floats. The folklore says 4× smaller and faster. Measure it.



On our CNN

64.4 KB → 64.6 KB.

Dynamic quantisation only converts Linear layers. This network's only Linear is a 32×4 head — nearly all the weights are in convolutions it never touches.



On a 790k-parameter MLP

3,087 KB → 779 KB (4.0x smaller).

But latency went from 0.07 ms to 0.09 ms. The int8 kernels were the slower path on this CPU.

Quantisation buys memory. It does not automatically buy speed. An engineer could have spent the sprint on int8 and gained nothing on either axis.

On the Robot: the Integration Pattern

In a ROS 2 system the model becomes a node that subscribes to a camera topic and publishes a verdict.

```
def on_image(self, msg):
    patches = to_patches(msg)

    # 1. GATE FIRST
    c = patches.std(axis=(2,3)).ravel()
    if np.mean((c<self.lo)|(c>self.hi)) > 0.10:
        self.publish("REJECT: out of distribution")
        return

    # 2. batch the whole part
    with torch.no_grad():
        pred = self.model(patches).argmax(1)
```

The gate runs first

and can veto the model

Patches are batched

the optimisation that mattered

The exported program loads directly

no model class on the robot

Also insist on: logging every input statistic and the prediction distribution; a conventional fallback when the gate fires; and a human who sees the rejects.

Where Simulated Data Comes From — and Further Reading



Generative models

Chapter 17 — GANs

Our simulator was hand-written. The alternative is to LEARN a generator from a small set of real images and synthesise the rest.

Same promise, same trap: a generator trained on one camera produces that camera's quirks.



Graph neural networks

Chapter 18 — further reading

For problems whose structure is a graph rather than a grid or a sequence: multi-robot coordination, kinematic chains, sensor networks.

Not examinable — but the natural next chapter for robotics.

The deployment lesson transfers: however you generate training data, the question is always whether its distribution covers the one you will meet.

This Week's Lab



Everything on these slides, measured by you. The whole notebook runs in about three minutes on a CPU.



Measure the gap

And show more data cannot close it



Write domain randomisation

Narrow versus wide, quantified



Catch it confidently wrong

Then build a monitor that works



Export and benchmark

Decide whether to optimise at all

What to Take Away

1. **A held-out test set does not predict deployment**

   It is drawn from the training distribution. The factory is not.

2. **More data from the wrong distribution is still wrong**

   Four times the simulated data moved real accuracy from 0.267 to 0.278.

3. **Randomisation helps exactly as far as it reaches**

   Narrow 0.497, wide 0.870. You are covering corruptions, not adding magic.

4. **Confidence is not a safety mechanism**

   Chance accuracy at 93% confidence. Softmax ranks classes; it never checks the input.

5. **Monitor the data, not the model's opinion of itself**

   One scalar per image flagged 99.8% of the shift, with no labels at all.

6. **Measure before you optimise**

   Batching bought 2.5×. Quantisation bought nothing, and the real fault was never speed.